CardioReport Robustness Specification

How the system automatically decides which report to generate, validates the data is good enough, finds the clinically interesting windows, and guarantees every report is trustworthy.

Contents

1. The Five Decisions the System Must Make Automatically

Right now, a human (you) makes these decisions manually. For the product to scale, the system must make them deterministically. Here is what separates a tool from a product:

Decision	What It Answers	Wrong Answer =
1. Data Validation	"Is the data clean enough to generate a credible report?"	Garbage in, garbage out. A report from 3 hours of noisy data destroys trust.
2. Report Type Selection	"Which report template fits this patient's available sensors?"	Showing bed data charts when no bed sensor exists. Fake signals.
3. Window Intelligence	"What time period should this report cover?"	A 30 day report where all the action happened in the last 3 days buries the story.
4. Phase Detection	"What is the clinical narrative arc within this window?"	Listing findings without temporal context. "9 episodes detected" vs "stable then crashed."
5. Signal Strength	"Is there enough clinical signal to justify generating this report?"	A GREEN report for a perfectly stable patient adds no value. Know when to say nothing.

2. Decision 1: Data Validation (Quality Gates)

Before generating any report, the system must check whether the data is trustworthy. These gates run in order. If any gate fails, the system either adjusts or refuses to generate.

Gate 1: Minimum coverage threshold

```
def validate_coverage(df, start, end, settings):
expected = (end - start).total_seconds() / 3600 + 1
recorded = len(df)
coverage = recorded / expected

if coverage < 0.30: # Less than 30%
return 'REJECT', 'Insufficient data coverage (below 30%)'
if coverage < 0.50: # 30-50%
return 'WARN', 'Low data coverage; interpret with caution'
return 'PASS', None
```

Gate 2: Minimum days with data

```
def validate_days(df, min_days=3):
days_with_data = df.groupby(df['Date'].dt.date).size()
days_with_enough = len(days_with_data[days_with_data >= 4]) # at least 4h per day

if days_with_enough < min_days:
return 'REJECT', f'Only {days_with_enough} days have sufficient data'
return 'PASS', None
```

Gate 3: Low confidence ratio

```
def validate_confidence(df, settings):
low_conf = (df['cnt'] < settings.low_confidence_cnt).sum()
ratio = low_conf / len(df) if len(df) else 1.0

if ratio > 0.50: # More than half the data is low confidence
return 'REJECT', 'Majority of readings are low confidence'
if ratio > 0.25:
return 'WARN', f'{ratio:.0%} of readings are low confidence'
return 'PASS', None
```

Gate 4: Vital sign range sanity

```
def validate_ranges(df):
issues = []
if df['avg_hr'].min() < 10 or df['avg_hr'].max() > 250:
issues.append('HR values outside physiologic range (10 to 250 bpm)')
if df['avg_rr'].min() < 2 or df['avg_rr'].max() > 60:
issues.append('RR values outside physiologic range (2 to 60 brpm)')
if df['avg_hr'].std() < 0.5: # Flatline = sensor stuck
issues.append('HR shows near zero variability (possible sensor malfunction)')

if issues:
return 'WARN', '; '.join(issues)
return 'PASS', None
```

Gate 5: Consecutive gap detection

```
def validate_gaps(df):
sorted_dates = df['Date'].sort_values()
gaps = sorted_dates.diff()
max_gap = gaps.max()
max_gap_hours = max_gap.total_seconds() / 3600 if pd.notna(max_gap) else 0

if max_gap_hours > 72: # 3 day gap
return 'WARN', f'Largest data gap: {max_gap_hours:.0f} hours'
return 'PASS', None
```

Combined quality gate orchestrator

```
def run_quality_gates(df, start, end, settings):
gates = [
validate_coverage(df, start, end, settings),
validate_days(df),
validate_confidence(df, settings),
validate_ranges(df),
validate_gaps(df),
]

rejects = [msg for status, msg in gates if status == 'REJECT']
warnings = [msg for status, msg in gates if status == 'WARN']

if rejects:
return {
'can_generate': False,
'reason': rejects[0],
'warnings': warnings,
}
return {
'can_generate': True,
'warnings': warnings, # passed to narrative for disclosure
}
```

> **Critical:** Warnings from the quality gates are passed into the narrative generator. If there are warnings, the narrative MUST include a data quality caveat sentence. If the gate rejects, the API returns a 422 with the reason instead of generating a report.

3. Decision 2: Report Type Selection (Sensor Routing)

The system must automatically determine which report template to use based on what data is available. This is the decision matrix:

Available Sensors	Date Overlap?	Report Types Offered	Default Selection
Chair only	N/A	Chair Weekly, Chair Monthly	Weekly if window <= 10 days, Monthly if > 10 days
Bed only	N/A	Bed Activity	Bed Activity (only option)
Chair + Bed	No overlap	Chair Weekly/Monthly + Bed Activity (separate)	Show both independently; no combined report
Chair + Bed	Yes, overlap exists	All 4 types including Combined Positional	Combined for overlap window; individual for full ranges

Auto selection logic when user clicks 'Generate' without choosing a type:

```
def auto_select_report_type(patient, start, end):
window_days = (end - start).days + 1

# If both sensors overlap in the requested window, prefer combined
if patient.has_both:
overlap = patient.overlapping_dates
if overlap and start >= overlap[0] and end <= overlap[1]:
return 'combined_positional'

# Bed only patient
if patient.has_bed and not patient.has_chair:
return 'bed_activity'

# Chair patient: weekly vs monthly based on window size
if patient.has_chair:
if window_days <= 10:
return 'chair_weekly'
else:
return 'chair_monthly'

return None # No report possible
```

4. Decision 3: Automatic Window Intelligence

This is the hardest problem. When a clinician says 'generate a report for this patient,' the system needs to pick the right time window. Too broad and the story is diluted. Too narrow and context is lost. Here is how to do it:

Strategy 1: Preset windows (user selectable)

These are the standard options the frontend should offer:

Preset	Window Size	Report Type	Use Case
Last 7 days	7 days	Weekly	Shift handoff, quick status check
Last 14 days	14 days	Biweekly	RPM review cycle, trend detection
Last 30 days	30 days	Monthly	Provider visit summary, billing documentation
Custom range	User defined	Auto selected	Focused investigation of a specific period
Most interesting week	Auto detected	Weekly	System picks the week with highest clinical signal

Strategy 2: 'Most interesting week' auto detection

This is the feature that makes the product feel intelligent. It scans the entire dataset and finds the 7 day window with the highest clinical signal:

```
def find_most_interesting_week(df, episodes, settings):
"""Slide a 7-day window across the data and score each position."""
dates = pd.date_range(df['Date'].min().normalize(),
df['Date'].max().normalize() - pd.Timedelta(days=6))
best_score = -1
best_start = None

for start in dates:
end = start + pd.Timedelta(days=6, hours=23, minutes=59)
w = df[(df['Date'] >= start) & (df['Date'] <= end)]
w_eps = [e for e in episodes
if pd.Timestamp(e['start']) >= start
and pd.Timestamp(e['start']) <= end]

if len(w) < 20: # Skip windows with too little data
continue

# Score this window
score = compute_window_score(w, w_eps)
if score > best_score:
best_score = score
best_start = start

return best_start, best_start + pd.Timedelta(days=6, hours=23, minutes=59)


def compute_window_score(w, episodes):
"""Score a time window by clinical interest."""
score = 0

# Episode burden (most important)
score += sum(e['severity_score'] for e in episodes) * 2

# Coupling (HR + RR concurrent = very interesting)
coupled = sum(1 for e in episodes if e.get('cooccur_brady_tachypnea'))
score += coupled * 10

# HR variability (wide P5 to P95 = interesting)
hr = w['avg_hr'].dropna()
if len(hr) > 10:
spread = hr.quantile(0.95) - hr.quantile(0.05)
if spread > 40: score += 15
elif spread > 25: score += 8

# Mixed episode types (brady + tachy in same week = very interesting)
types = set(e['condition'] for e in episodes)
if len(types) >= 3: score += 12
elif len(types) >= 2: score += 5

# Duration bonus (long episodes more interesting than short)
max_dur = max((e['hours'] for e in episodes), default=0)
if max_dur >= 6: score += 10
elif max_dur >= 3: score += 5
```

```
return score
```

5. Decision 4: Phase Detection (Finding the Clinical Story)

The 10/10 reports we built told a temporal story: 'Phase 1: low HR with coupling, Phase 2: HR surge, Phase 3: signal fading.' That was done manually. Here is how to automate it:

Phase detection algorithm

```
def detect_phases(daily_df, episodes):
"""Segment the monitoring window into clinically distinct phases."""
phases = []
n = len(daily_df)
if n < 3:
return [{'label': 'Full period', 'days': list(range(n)), 'type': 'single'}]

# Compute daily episode burden
daily_df = daily_df.copy()
daily_df['ep_score'] = 0
daily_df['ep_types'] = ''

for ep in episodes:
ep_start = pd.Timestamp(ep['start']).normalize()
ep_end = pd.Timestamp(ep['end']).normalize()
mask = (daily_df['date'] >= ep_start) & (daily_df['date'] <= ep_end)
daily_df.loc[mask, 'ep_score'] += ep['severity_score']

# Classify each day
for i, row in daily_df.iterrows():
if row['ep_score'] == 0:
daily_df.loc[i, 'day_class'] = 'stable'
elif row['hr_avg'] < 55:
daily_df.loc[i, 'day_class'] = 'low_hr'
elif row['hr_avg'] > 85:
daily_df.loc[i, 'day_class'] = 'high_hr'
else:
daily_df.loc[i, 'day_class'] = 'mixed'

# Group consecutive days of the same class into phases
current_class = daily_df.iloc[0]['day_class']
phase_start = 0

for i in range(1, n):
if daily_df.iloc[i]['day_class'] != current_class:
phases.append({
'type': current_class,
'start_idx': phase_start,
'end_idx': i - 1,
'days': list(range(phase_start, i)),
})
current_class = daily_df.iloc[i]['day_class']
phase_start = i

# Final phase
phases.append({
'type': current_class,
'start_idx': phase_start,
```

```
'end_idx': n - 1,
'days': list(range(phase_start, n)),
})

# Label phases
PHASE_LABELS = {
'stable': 'Stable',
'low_hr': 'Low HR + Coupling',
'high_hr': 'HR Surge',
'mixed': 'Mixed Instability',
}
for i, p in enumerate(phases):
start_date = daily_df.iloc[p['start_idx']]['date']
end_date = daily_df.iloc[p['end_idx']]['date']
p['label'] = f'Phase {i+1}: {PHASE_LABELS.get(p["type"], p["type"])}'
p['date_range'] = f'{start_date.strftime("%b %d")} to {end_date.strftime("%b %d")}'

return phases
```

How phases feed into the narrative:

```
def build_phase_narrative(phases, daily_df, episodes):
"""Auto generate the phase-by-phase narrative."""
parts = []

if len(phases) == 1 and phases[0]['type'] == 'stable':
parts.append('Vital signs remained stable throughout the monitoring period.')
return ' '.join(parts)

parts.append(f'The monitoring period showed {len(phases)} distinct phases.')

for phase in phases:
days = daily_df.iloc[phase['days']]
avg_hr = days['hr_avg'].mean()
phase_eps = [e for e in episodes
if pd.Timestamp(e['start']).normalize() >= days['date'].min()
and pd.Timestamp(e['start']).normalize() <= days['date'].max()]

if phase['type'] == 'stable':
parts.append(
f'{phase["label"]} ({phase["date_range"]}): '
f'Heart rate averaged {avg_hr:.0f} bpm with no episodic events.'
)
elif phase['type'] == 'low_hr':
coupled = sum(1 for e in phase_eps if e.get('cooccur_brady_tachypnea'))
min_hr = days['hr_min'].min()
parts.append(
f'{phase["label"]} ({phase["date_range"]}): '
f'Heart rate dropped to lows of {min_hr:.0f} bpm'
+ (f' with {coupled} episodes showing concurrent elevated breathing rate.'
if coupled else '.')
)
elif phase['type'] == 'high_hr':
max_hr = days['hr_max'].max()
```

```
    parts.append(
    f'{phase["label"]} ({phase["date_range"]}): '
    f'Heart rate rose with daily averages reaching {avg_hr:.0f} bpm '
    f'and peaks near {max_hr:.0f} bpm.'
    )

    return ' '.join(parts)
```

6. Decision 5: Signal Strength Assessment

Not every report is worth generating. A GREEN report for a perfectly stable patient wastes physician time. The system should indicate how 'interesting' a report is:

```
def compute_report_priority(episodes, quality_result, phases):
    """Classify the report as HIGH / MEDIUM / LOW priority."""
    if not quality_result['can_generate']:
        return 'SKIP', 'Data quality insufficient'

    if not episodes:
        return 'LOW', 'No episodic events detected; stable patient'

    max_sev = max(e['severity_score'] for e in episodes)
    coupled = any(e.get('cooccur_brady_tachypnea') for e in episodes)
    phase_count = len([p for p in phases if p['type'] != 'stable'])

    # HIGH: urgent clinical signal
    if max_sev >= 9 or coupled or phase_count >= 2:
        return 'HIGH', 'Significant clinical events detected'

    # MEDIUM: notable but not urgent
    if max_sev >= 5 or len(episodes) >= 3:
        return 'MEDIUM', 'Episodic events warrant review'

    # LOW: minimal signal
    return 'LOW', 'Minor events only; routine monitoring'
```

How priority affects the workflow:

Priority	Action	Frontend Display
HIGH	Generate immediately. Send notification if automated.	Red badge. Report appears at top of queue.
MEDIUM	Generate on request or scheduled run.	Amber badge. Available in patient list.
LOW	Generate only if user explicitly requests.	Green badge. "No significant findings" summary shown.
SKIP	Do not generate. Show data quality warning.	Gray badge. "Insufficient data" message with reason.

7. The Complete Decision Pipeline

When a report generation request comes in, here is the exact sequence of operations:

Step	Operation	Output	Can Fail?
1	Look up patient in registry	PatientRecord with sensors list	404 if not found
2	Select report type (auto or user chosen)	report_type string	400 if no sensors match
3	Load hourly data from correct sensor(s)	DataFrame(s) with sensor tag	500 if file corrupt
4	Run quality gates	PASS/WARN/REJECT + messages	422 if REJECT
5	Compute stats + daily aggregates	Stats dict + daily DataFrame	No
6	Detect episodes	Episodes list with scores	No
7	Compute triage + trend + action posture	Three deterministic labels	No
8	Detect phases	Phase list with labels and date ranges	No
9	Compute report priority	HIGH/MEDIUM/LOW/SKIP	No
10	Generate narrative (taxonomy or AI)	Narrative text + action bullets	Fallback to taxonomy
11	Generate charts	PNG buffers (candlestick, histogram, bed hours)	No
12	Render PDF	One page PDF bytes	No
13	Return response	PDF + JSON metadata (priority, warnings, phases)	No

> **The pipeline is fail safe at every step.** Patient not found = 404. No sensors = 400. Data too sparse = 422 with reason. AI fails = taxonomy fallback. File corrupt = 500 with stack trace. The system never silently generates a bad report.

8. Edge Cases and How to Handle Them

Edge Case	How to Detect	How to Handle
Sensor stuck (flatline HR)	std(avg_hr) < 0.5 over 24+ hours	WARN in quality gate. Add "possible sensor malfunction" to narrative.
Patient moved between rooms	Both sensors active in same hour	Use the sensor with higher sample count (cnt) for that hour.
No data for requested window	len(df) == 0 after filtering	Return 422: "No data available for this date range." Suggest available dates.
Bed file missing Summary sheet	has_summary == False	Generate bed report without bed hours chart. Note in footer: "Bed time data unavailable."
Extreme outlier (HR = 300)	validate_ranges catches it	Clip to physiologic range (30 to 220 bpm) before analysis. Add warning.
All episodes are S0 (low severity)	max severity < 5	Generate GREEN report with "No significant findings" narrative.
50+ episodes in window	len(episodes) > 50	Events table still caps at 6 to 8 rows. Narrative says "numerous episodes" not exact count.
Two files for same patient/sensor	Same patient_id + sensor_type in registry	Concatenate DataFrames, deduplicate by Date, keep row with higher cnt.
June 30 has only 3 hours (patient dying)	Daily coverage < 4 hours on last day	Flag in narrative: "Monitoring signal declined on [date]. Clinical follow up advised."

9. Testing Matrix: Every Scenario Your System Must Pass

Run each of these tests. If any fails, the system is not ready for production:

#	Test Scenario	Input	Expected Result
1	Chair patient, stable week	934297 0122, May 1 to 7	GREEN report, 0 episodes, "stable" narrative
2	Chair patient, critical week	934297 0122, Jun 24 to 30	RED report, 9 episodes, 3 phase narrative, coupled flags
3	Chair patient, full month	934297 0122, Jun 1 to 30	RED report, weekly progression table, "stable then crashed" narrative
4	Bed patient, full period	934297 0134, Oct 20 to Nov 20	Bed Activity report with bed hours chart, alert correlation
5	Insufficient data	934297 0122, Jun 30 only	422 rejection: coverage below 30%
6	Wrong sensor requested	934297 0122, bed_activity report	400 error: "No bed sensor available for this patient"
7	Auto window selection	934297 0122, "most interesting week"	System selects Jun 22 to 28 window (highest clinical signal)
8	Same report, same data, twice	934297 0122, Jun 24 to 30, run twice	Byte identical PDF output (deterministic)
9	Threshold change cascade	Change brady from 50 to 48, rerun test 2	Fewer episodes, different triage, chart lines move to 48
10	AI fallback	USE_LLM=True with invalid API key	Report generates with taxonomy narrative, no error

Test 8 is the most important. If the same input produces different output on two runs, you have a non deterministic bug somewhere. For USE_LLM=false, the output must be byte identical. For USE_LLM=true, the numbers, triage, scores, and charts must be identical; only the narrative text may vary.

The definition of "robust": A robust system is one where a developer can drop any Excel file into the data directory, restart the server, and the patient automatically appears in the UI with the correct sensor types, available report types, and date ranges. Clicking "Generate" produces a clinically credible report or a clear error message explaining why it cannot. No manual configuration. No hardcoded patient IDs. No assumptions about file structure. That is the target.